

# Graph

# Graph is

- A data structure that consists of a set of nodes (vertices) and a set of edges that relate the nodes to each other.
- The set of edges describes relationships among the vertices.
- A graph  $G$  is defined as follows:
- $G=(V,E)$
- $V(G)$ : a finite, nonempty set of vertices
- $E(G)$ : a set of edges (pairs of vertices)

# Types

|            | UNDIRECTED | DIRECTED |
|------------|------------|----------|
| UNWEIGHTED |            |          |
| WEIGHTED   |            |          |

# Terminologies

- Adjacent nodes: two nodes are adjacent if they are connected by an edge
- Path: a sequence of vertices that connect two nodes in a graph
- A simple path is a path in which all vertices, except possibly in the first and last, are different.
- Complete graph: a graph in which every vertex is directly connected to every other vertex.
- A cycle is a simple path with the same start and end vertex.

# Terminologies

- The degree of vertex  $v$  is the no. of edges incident on vertex.

# Terminologies

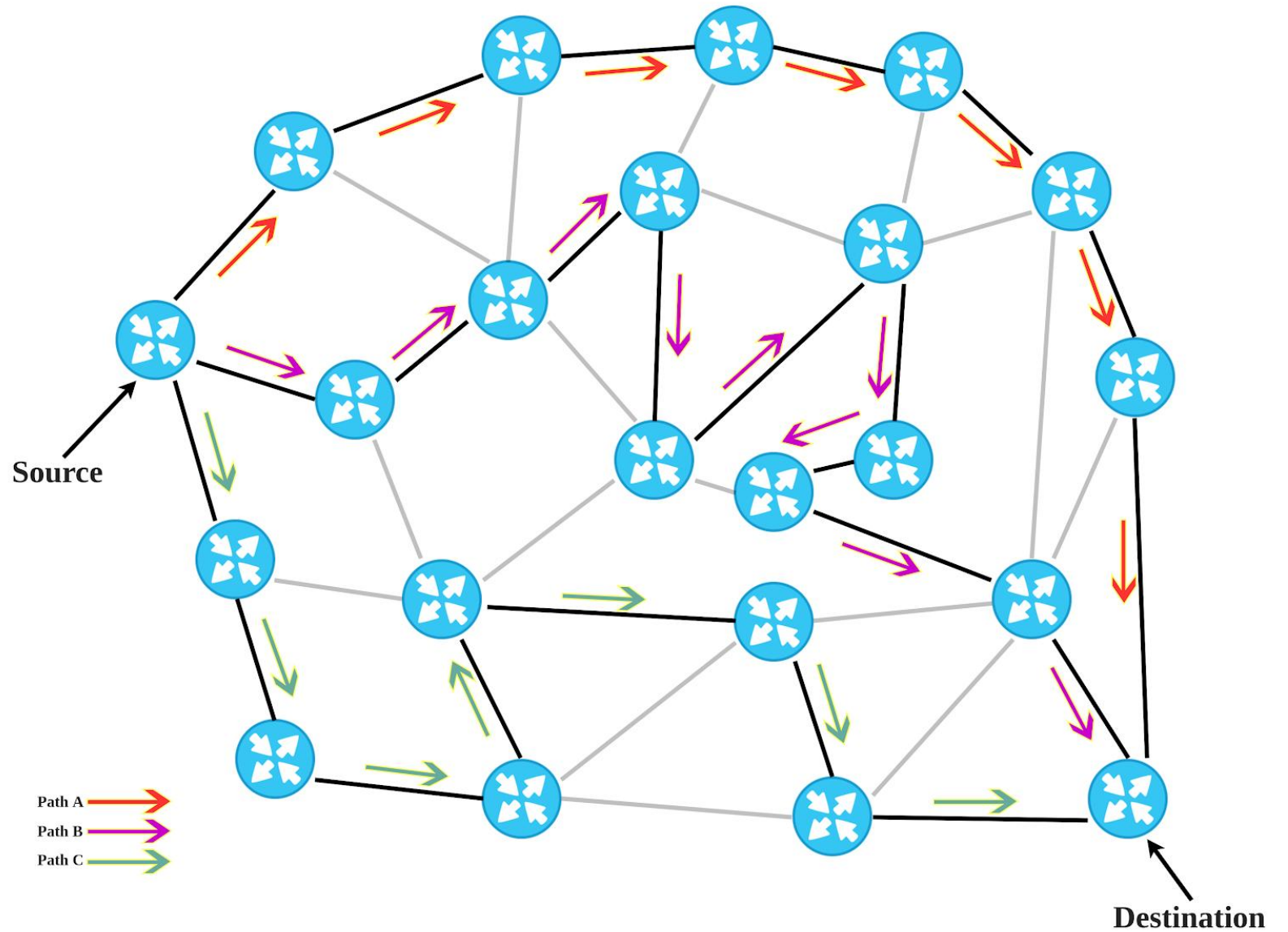
- A complete graph has an edge between every pair of vertices.
- Loops: edges that connect a vertex to itself
- Paths: sequences of vertices  $p_0, p_1, \dots, p_m$  such that each adjacent pair of vertices are connected by an edge.
- A simple path is a path in which all vertices, except possibly in the first and last, are different.

# Terminologies

- Multiple Edges: two nodes may be connected by  $>1$  edge
- Simple Graphs: have no loops and no multiple edges
- In-degree of vertex  $i$  is the number of edges incident to  $i$  (i.e., the number of incoming edges).
- Out-degree of vertex  $j$  is the number of edges incident from  $j$  (i.e., the number of outgoing edges).

# Applications

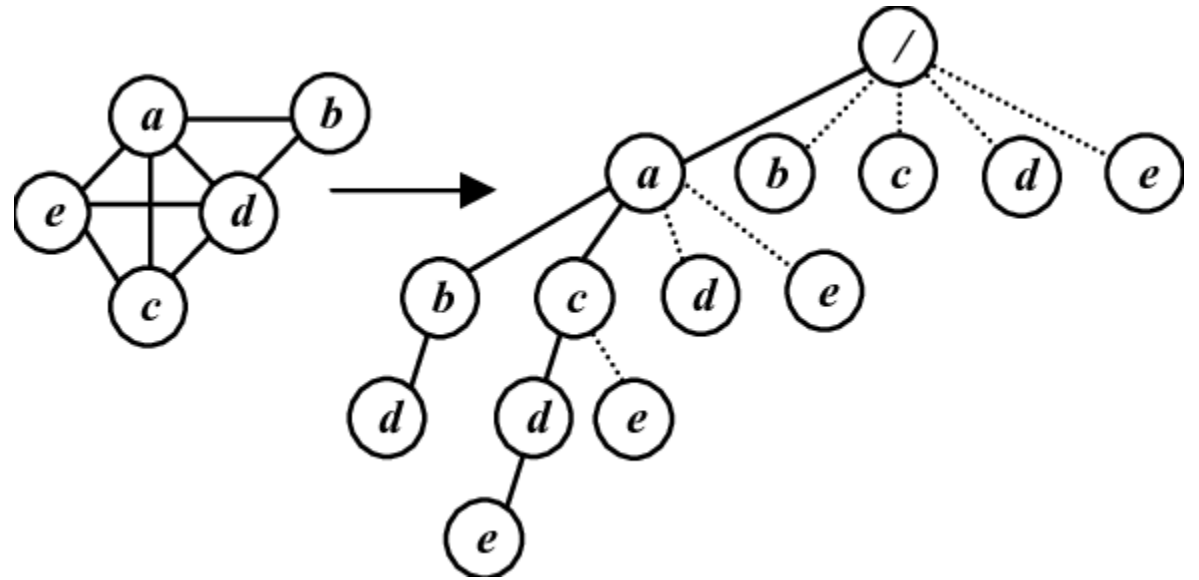
- In Routing



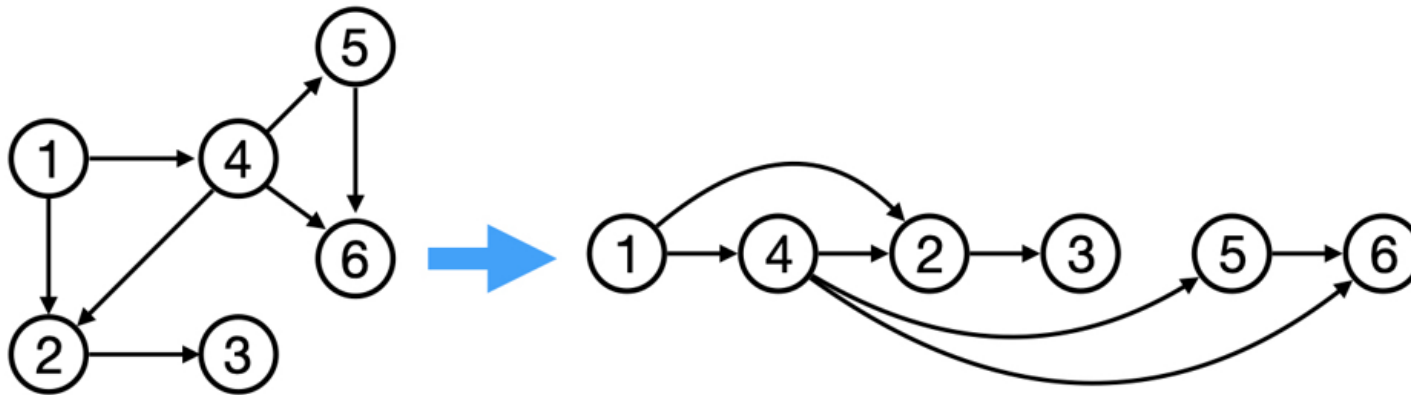
- In biometric



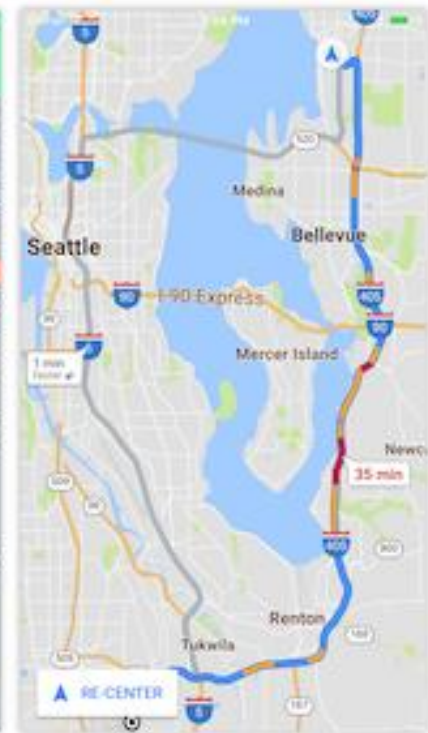
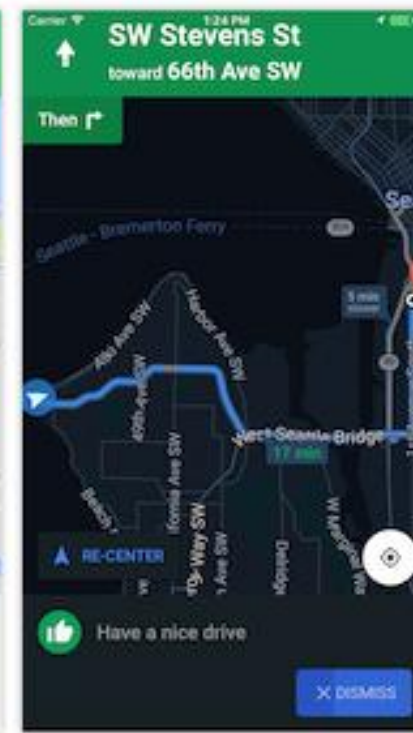
- Searching in graph



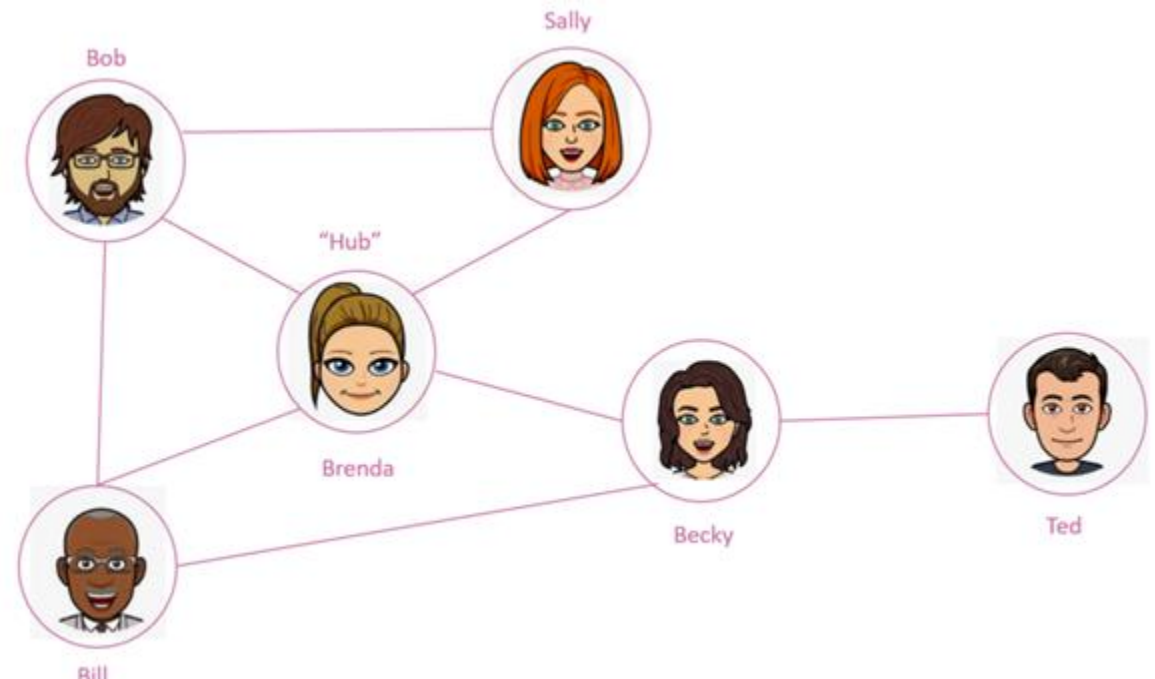
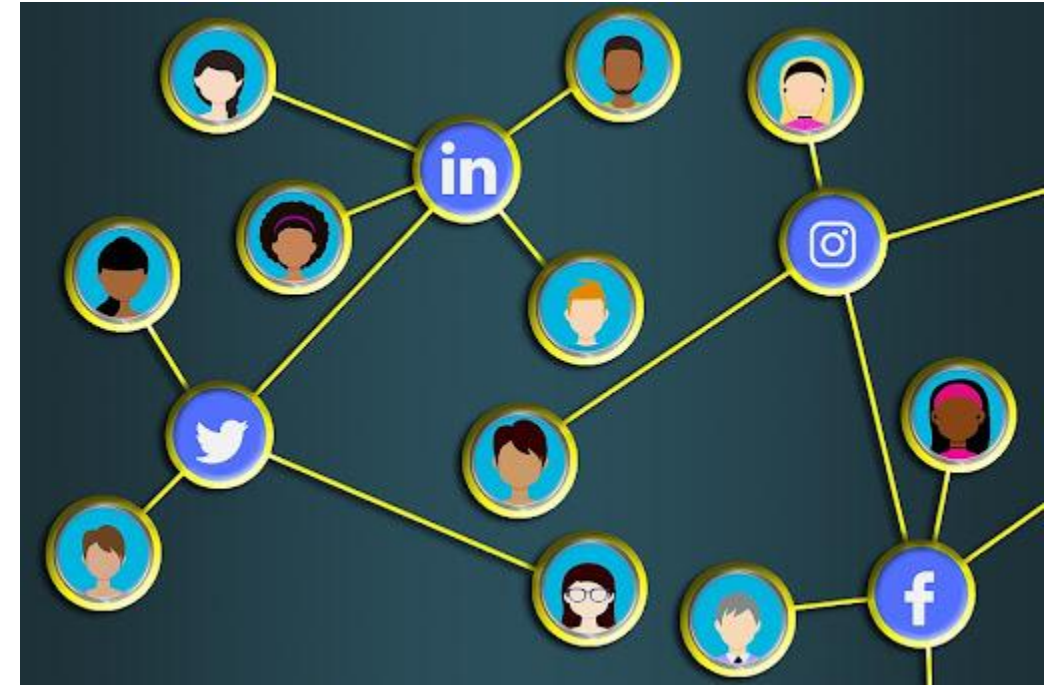
- Handling flow problems
- Topological sort



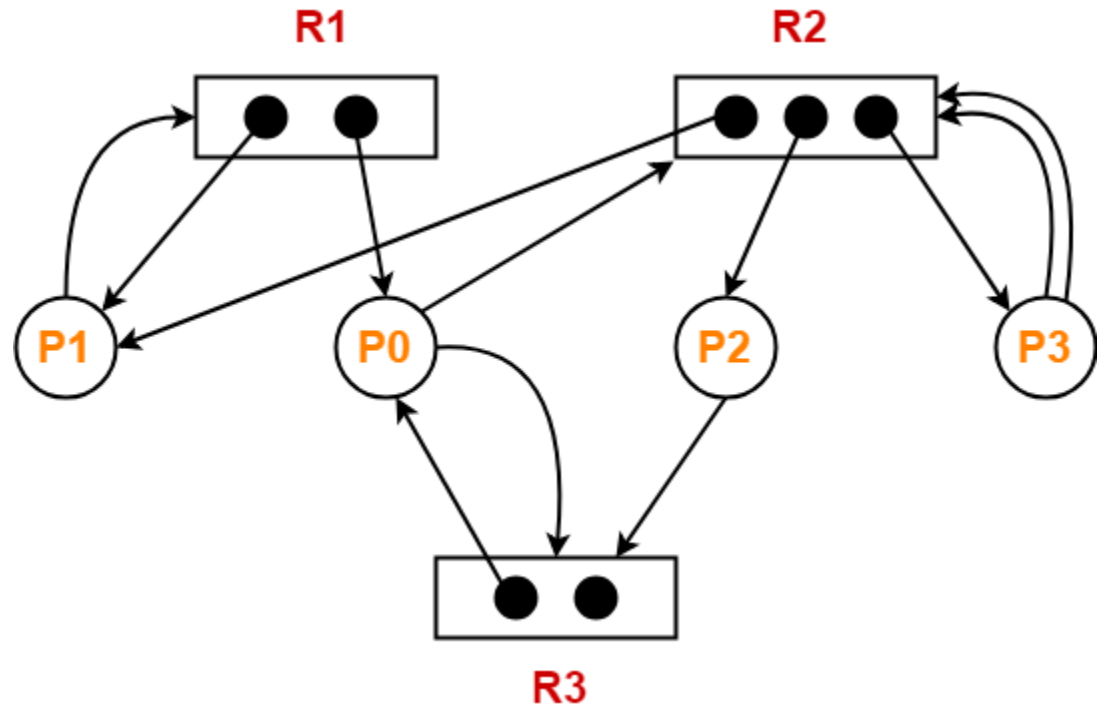
- **Google maps** uses graphs for building transportation systems, where intersection of two(or more) roads are considered to be a vertex and the road connecting two vertices is considered to be an edge, thus their navigation system is based on the algorithm to calculate the shortest path between two vertices.



- In **Facebook**, users are considered to be the vertices and if they are friends then there is an edge running between them. Facebook's Friend suggestion algorithm uses graph theory. Facebook is an example of **undirected graph**.



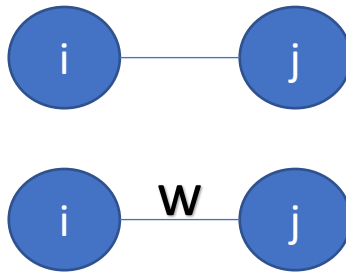
- In **Operating System**, we come across the Resource Allocation Graph where each process and resources are considered to be vertices. Edges are drawn from resources to the allocated process, or from requesting process to the requested resource. If this leads to any formation of a cycle then a deadlock will occur.



# Graph representation

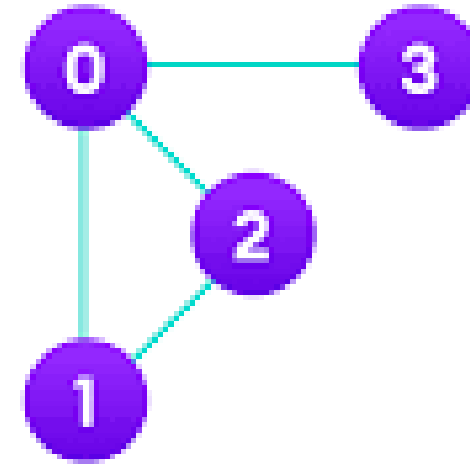
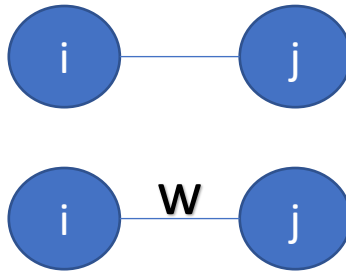
- Adjacency matrix

- $V \times V$  matrix(storage of  $v \times v$ )
- Static
- Easy
- To fill if unweighted
- $\text{AdjMat}[i][j]=1/0$
- To fill if weighted
- $\text{AdjMat}[i][j]=w/\infty$



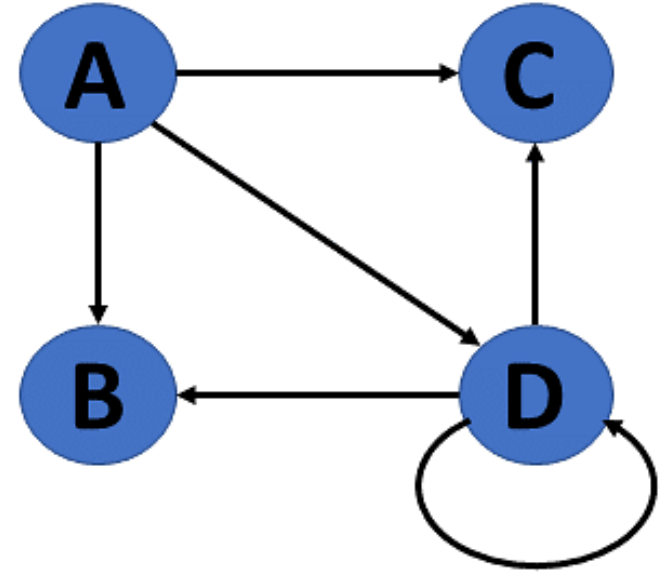
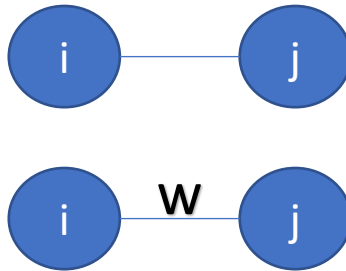
# Graph representation

- Adjacency matrix
  - $V \times V$  matrix
  - Static
  - Easy
  - To fill if unweighted
  - $\text{AdjMat}[i][j]=1/0$
  - To fill if weighted
  - $\text{AdjMat}[i][j]=w/\infty$



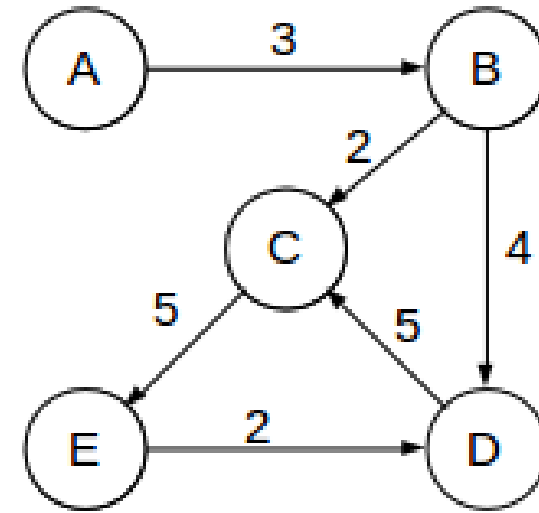
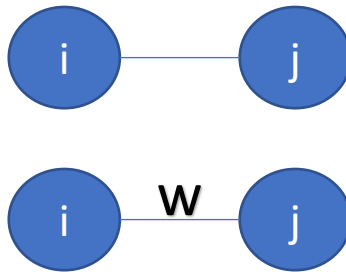
# Graph representation

- Adjacency matrix
  - $V \times V$  matrix
  - Static
  - Easy
  - To fill if unweighted
  - $\text{AdjMat}[i][j]=1/0$
  - To fill if weighted
  - $\text{AdjMat}[i][j]=w/\infty$



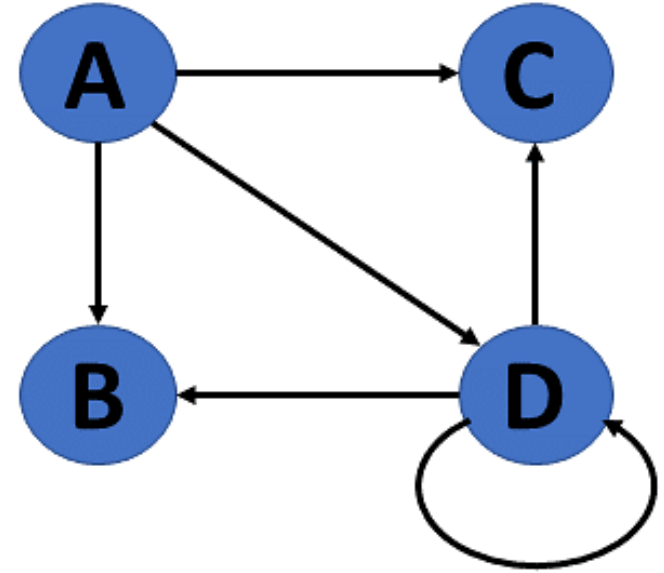
# Graph representation

- Adjacency matrix
  - $V \times V$  matrix
  - Static
  - Easy
  - To fill if unweighted
  - $\text{AdjMat}[i][j]=1/0$
  - To fill if weighted
  - $\text{AdjMat}[i][j]=w/\infty$



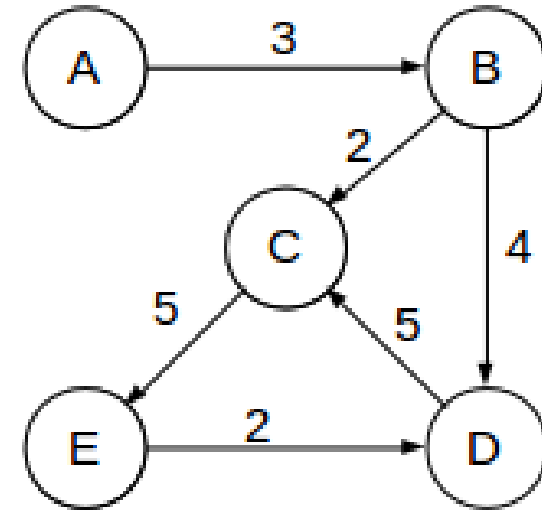
# Graph representation

- Adjacency List
  - Array of linked list
  - Each node is an edge
  - Can grow or sharing with need
  - Storage of  $(v+e)$



# Graph representation

- Adjacency List
  - Array of linked list
  - Each node is an edge
  - Can grow or sharing with need

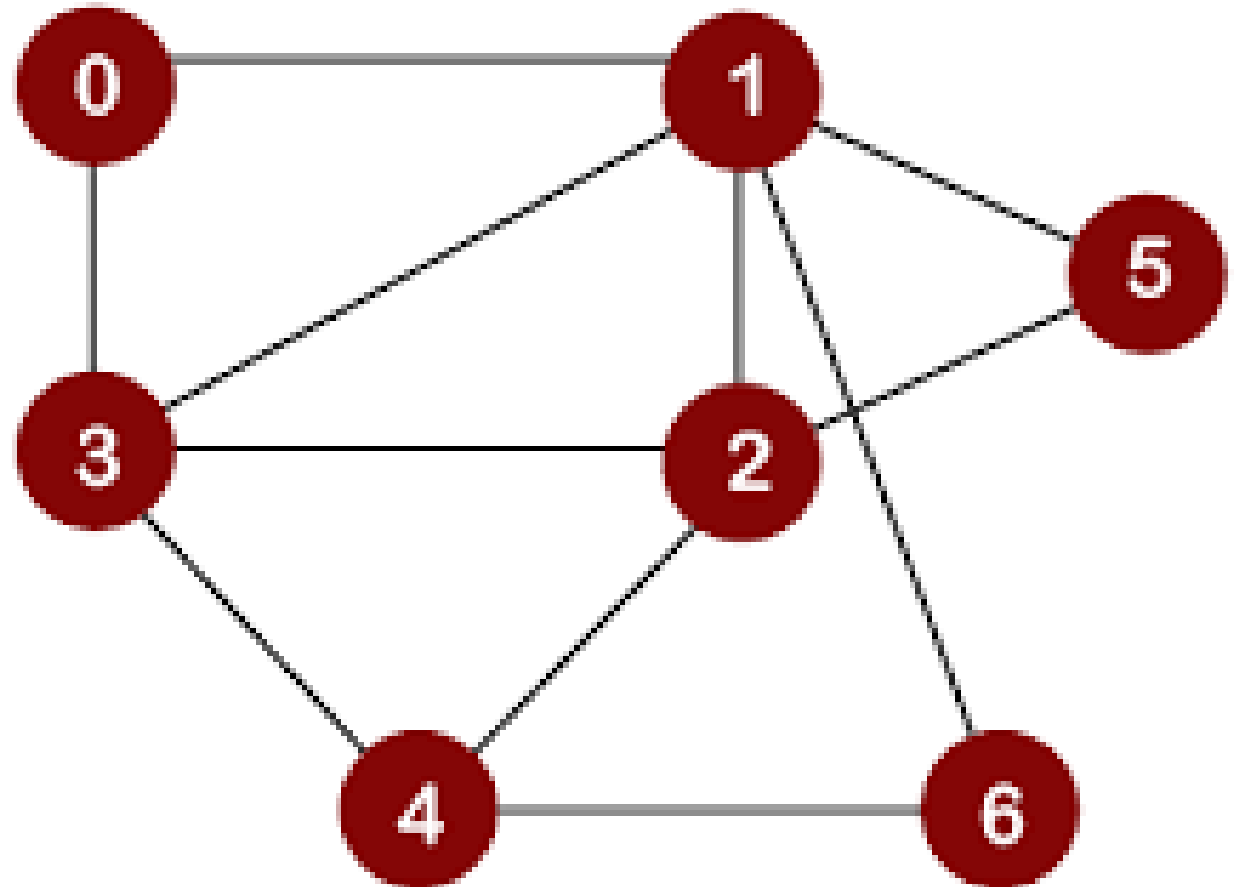


# Create a Graph

|          | <b>1</b> | <b>2</b> | <b>3</b> | <b>4</b> | <b>5</b> |
|----------|----------|----------|----------|----------|----------|
| <b>1</b> | 0        | 1        | 1        | 1        | 0        |
| <b>2</b> | 1        | 0        | 0        | 1        | 0        |
| <b>3</b> | 1        | 0        | 0        | 1        | 1        |
| <b>4</b> | 1        | 1        | 1        | 0        | 1        |
| <b>5</b> | 0        | 0        | 1        | 1        | 0        |

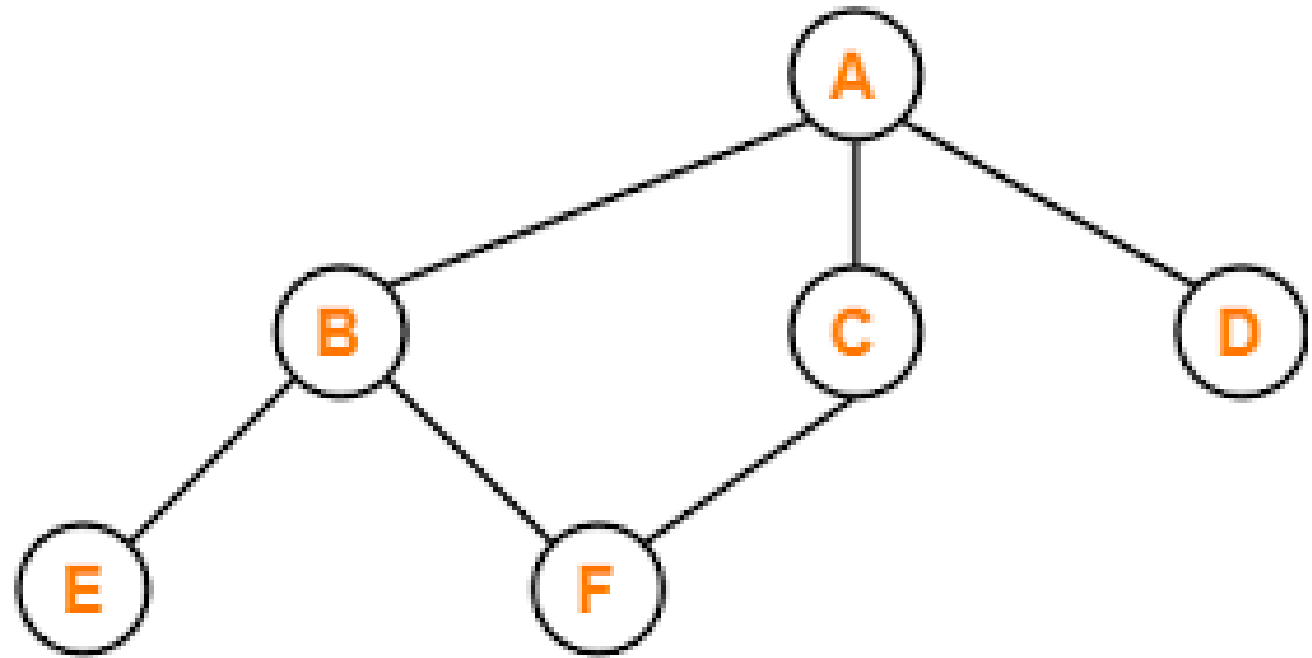
# Graph Searching

- BFS
- DFS



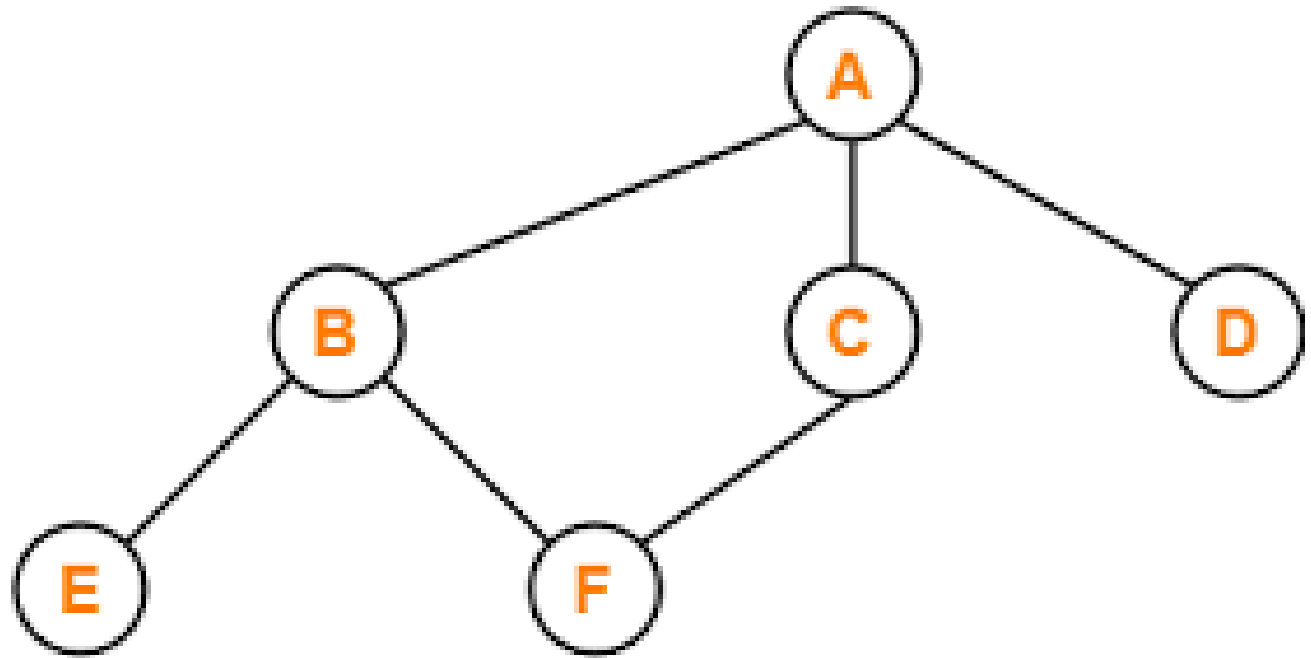
# BFS

1. Start by putting source on queue and marking it visited.
2. Search for all unvisited neighbors of element at queue front ,mark them visited and put them on queue.
3. Remove element at queue front
4. Continue 2,3 till queue is not empty



# DFS

- The DFS algorithm works as follows:
  1. Accept source ,mark it visited and push it on stack.
  2. Search any one unvisited neighbor of element at top of stack, mark it visited, push it on stack.
  3. If no unvisited element found then pop element from stack at go to step 2.
  4. Continue step 2,3 till stack is not empty

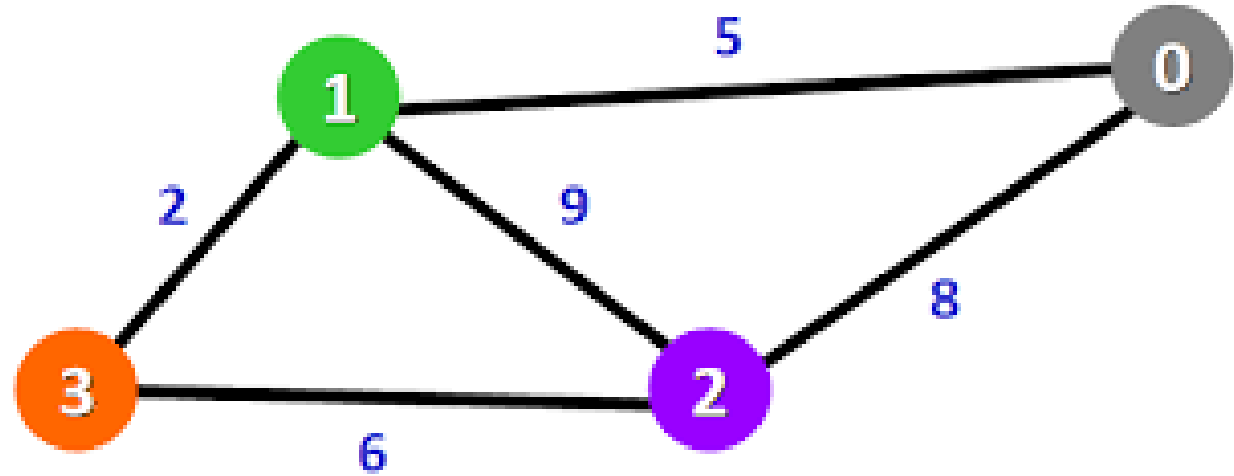


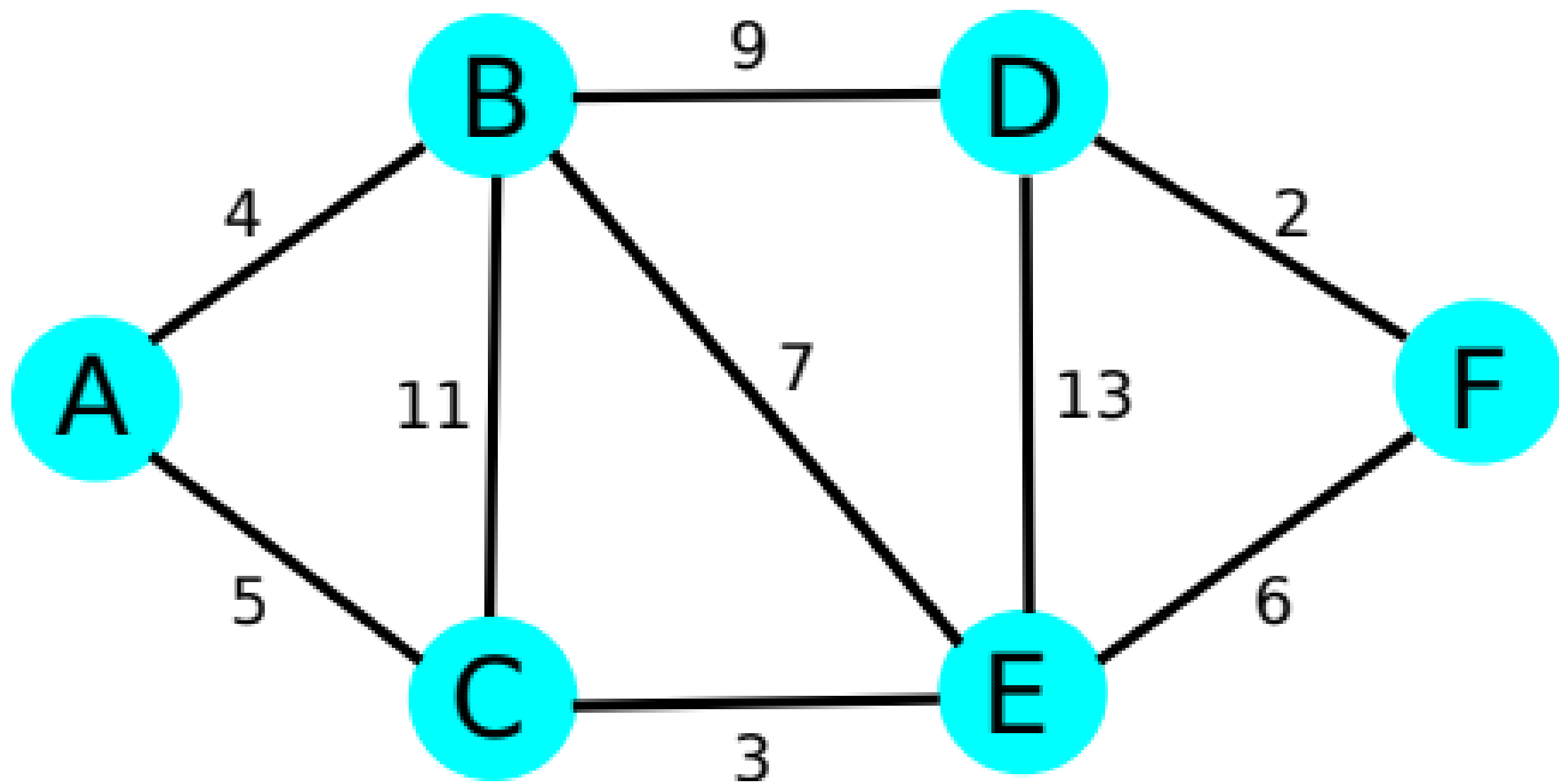
# Dijkstra's Shortest Path Algorithm

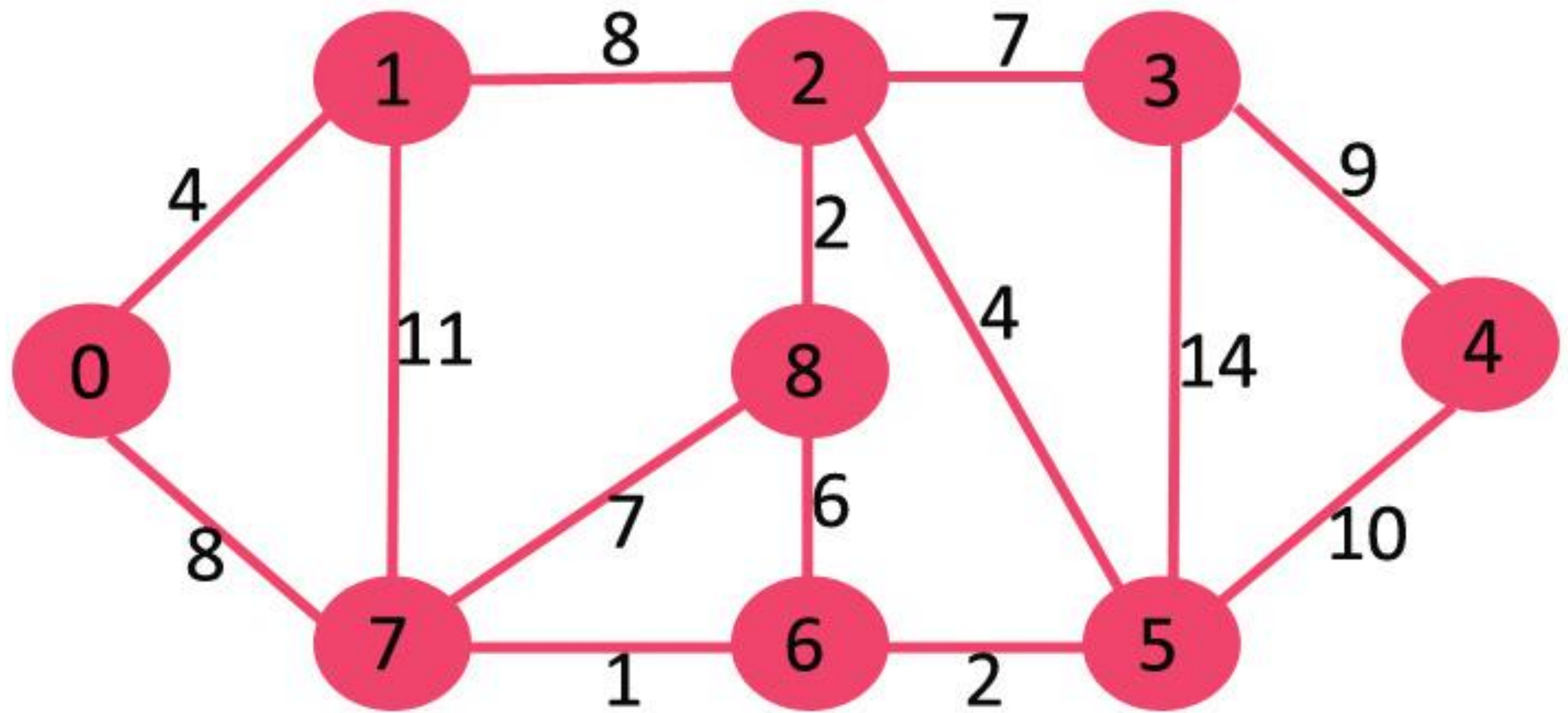
- Assign a distance value to all vertices in the input graph. Initialize all distance values as **INFINITE**. Assign the distance value as 0 for the source vertex so that it is picked first.
- While **sol** doesn't include all vertices
  - Pick a vertex **u** which is not there in **sol** and has a minimum distance value.
  - Include **u** to **sol**.
  - Then update distance value of all adjacent vertices of **u**.
    - To update the distance values, iterate through all adjacent vertices.
    - For every adjacent vertex **v**, if the sum of the distance value of **u** (from source) and weight of edge **u-v**, is less than the distance value of **v**, then update the distance value of **v**.

# Dijkstra's Shortest Path Algorithm

- Assign a distance value to all vertices in the input graph. Initialize all distance values as **INFINITE**. Assign the distance value as 0 for the source vertex so that it is picked first.
- While **sol** doesn't include all vertices
  - Pick a vertex **u** which is not there in **sol** and has a minimum distance value.
  - Include **u** to **sol**.
  - Then update distance value of all adjacent vertices of **u**.
    - To update the distance values, iterate through all adjacent vertices.
    - For every adjacent vertex **v**, if the sum of the distance value of **u** (from source) and weight of edge **u-v**, is less than the distance value of **v**, then update the distance value of **v**.







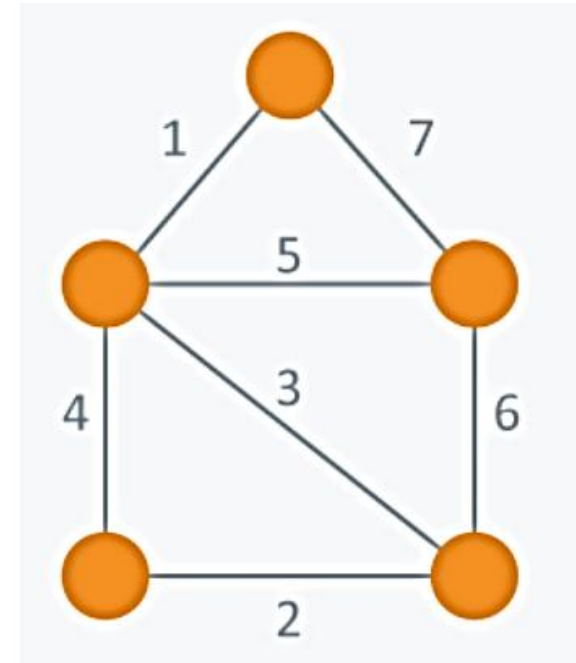
# Minimum Spanning Tree

- Given an undirected and connected graph  $G$ , a spanning tree of the graph is a tree that spans  $G$  (that is, it includes every vertex of  $G$ ) and is a subgraph of  $G$  (every edge in the tree belongs to  $G$ ) with minimum cost.

# Kruskal's Algorithm

- **Algorithm Steps:**

1. Accept graph  $G(V,E)$
2. Create sorted list of all edges on basis of weight
3. Till all nodes are not in sol do
  1. Extract an edge from sorted list
  2. Check if not forming cycle then accept in sol else reject and go to step 3



# Prim's Algorithm

- **Algorithm Steps:**
- Start from given source add it to sol tree.
- While all vertices are not in sol tree do
  - Select the cheapest edge between vertex of sol tree and non-sol vertex iff not forming cycle.

